CRM DE ATENDIMENTO E ORÇAMENTOS

# Entendendo o Backend

## Parte 2 — Casos de uso

Quem usa as regras do domínio para fazer algo útil — ainda sem banco de dados

1 2 3 4 5

Parte 2 de 5 do backend

VISÃO GERAL

# Onde estamos no plano do backend

1

### Fundação + Domínio

Regras do negócio em Python puro, com testes.

✓ CONCLUÍDA

2

### Casos de uso

As ações do usuário, orquestrando o domínio.

VOCÊ ESTÁ AQUI

3

### Persistência

PostgreSQL, SQLAlchemy e Alembic (migrations).

4

### API REST

Rotas FastAPI que o frontend vai chamar.

5

### Autenticação

Login com JWT e gestão de usuários.

A Parte 1 criou as regras (o domínio). A Parte 2 cria quem usa essas regras para executar as ações do usuário: os casos de uso.

Ainda sem banco de dados: tudo roda em memória. Os 160 testes do backend terminam em menos de meio segundo.

CONCEITO

# O que é um caso de uso?

É o roteiro de uma ação que o usuário faz no sistema: “registrar novo contato”, “mover o card”, “ver o painel”. Cada caso de uso da documentação (UC01, UC03...) virou uma classe.

*Analogia: o domínio é o livro de regras do futebol; o caso de uso é o juiz conduzindo a jogada. O juiz não inventa regra — ele aplica a que está no livro.*

**O roteiro quase sempre é o mesmo:**

1. **Carrega**
   Busca o orçamento (ou cliente) no repositório.
2. **Delega**
   Pede para o próprio objeto do domínio fazer a mudança — é ele quem decide se pode.
3. **Salva**
   Só se o domínio aceitou, grava de volta no repositório.

Regra de ouro da camada application/: o caso de uso orquestra, não decide. Nenhuma regra de negócio mora aqui — se um caso de uso tivesse um “if” sobre estágio, a regra estaria no lugar errado.

O TRUQUE CENTRAL

# Inversão de dependência: a tomada e os plugues




### 1 A tomada

A interface define o formato: salvar, buscar_por_id, listar_todos... Fica no domínio, que diz o que precisa.

### 2 Os plugues

Cada implementação encaixa nesse formato: em memória hoje, PostgreSQL na Parte 3.

### 3 O ganho

Trocar o banco não muda uma linha dos casos de uso. E os testes rodam sem banco (RNF06).

# A interface: um contrato sem implementação

```python
from abc import ABC, abstractmethod

class OrcamentoRepository(ABC):

    @abstractmethod
    def salvar(self, orcamento: Orcamento) -> None: ...

    @abstractmethod
    def buscar_por_id(self, orcamento_id: UUID) -> Orcamento | None: ...

    @abstractmethod
    def listar_todos(self) -> list[Orcamento]: ...

    @abstractmethod
    def listar_por_cliente(self, cliente_id: UUID) -> list[Orcamento]: ...
```

Arquivos em src/domain/repositories/. O “...” no lugar do corpo significa: aqui não há código, só a assinatura.

1 **ABC**

Classe abstrata: não dá para usar direto, só serve de molde.

2 **`@abstractmethod`**

Quem implementar é obrigado a escrever esse método — senão o Python recusa criar o objeto.

3 **Uma por agregado**

Cliente, Orcamento e Usuario. Interação não tem repositório: é salva junto com o Orçamento.

# Repositórios em memória: um banco de mentira, mas honesto

```
class _Armazem[T]:
    def salvar(self, item: T) -> None:
        self._dados[item.id] = deepcopy(item)    # guarda uma CÓPIA
        self.salvamentos += 1                    # conta as gravações

    def _buscar(self, item_id: UUID) -> T | None:
        item = self._dados.get(item_id)
        return deepcopy(item) if item else None   # devolve outra CÓPIA
```

### Por que cópias?

Num banco real, mexer no objeto carregado não muda o banco até salvar. As cópias imitam isso.

### Para que o contador?

salvamentos prova que nada foi gravado quando o domínio recusou (RNF04).

### Por que fica em src/adapters/ e não só nos testes?

É uma implementação de verdade da interface (um adapter). Dá para rodar o sistema inteiro sem banco, e os 9 testes de contrato em tests/unit/adapters/ (“salvar e buscar devolve igual”, “alterar sem salvar não persiste”...) vão rodar também contra o PostgreSQL na Parte 3 — garantindo que os dois plugues se comportam igual.

O QUE FOI CRIADO

# Os 8 casos de uso

UC01 · RF01, RF09

**`RegistrarNovoContato`**

Cria o Cliente e o primeiro Orçamento em NOVO_CONTATO.

UC03 · RF02, RF03

**`MoverEstagio`**

Muda o estágio se o Orçamento aceitar.

UC05 · RF03

**`DefinirValor`**

Define o valor e libera Enviado/Fechado.

UC06 · RF04

**`RegistrarInteracao`**

Adiciona mensagem, ligação ou reunião.

UC07 · RF05

**`ConsultarOrcamentos`**

Monta o Kanban: 5 colunas de cards.

UC08 · RF06

**`ConsultarMetricas`**

Orçamentos em aberto e taxa de fechamento.

UC09 · RF08

**`ConsultarCliente`**

O cliente com todos os seus orçamentos.

UC09 · RF07

**`ConsultarHistoricoCliente`**

Linha do tempo das interações do cliente.

Linha de cima: comandos (mudam dados). Linha de baixo: consultas (só leem). No código, todos terminam com UseCase.

EXEMPLO 1

# RegistrarNovoContato: valida tudo antes de gravar

```
def executar(self, dados: DadosNovoContato):
    agora = self._relogio()

    # 1) valida tudo construindo os objetos
    cliente = Cliente.cadastrar(
        nome=dados.nome,
        telefone=Telefone(dados.telefone),
        email=Email(dados.email),
        agora=agora,
    )
    orcamento = cliente.abrir_orcamento(hoje=agora.date())

    # 2) só então grava
    self._clientes.salvar(cliente)
    self._orcamentos.salvar(orcamento)
```

**✓ Dados válidos**

Cliente e Orçamento salvos; o card aparece em NOVO_CONTATO.

**✗ Telefone, e-mail ou nome inválido**

FormatoInvalido é lançado no passo 1, o passo 2 nunca roda e nada é salvo.

**O teste que prova:**

```
with pytest.raises(FormatoInvalido):
    use_case.executar(invalidos)
assert clientes.salvamentos == 0
```

Nunca fica “meio cadastro”: a validação vem antes da primeira gravação. Na Parte 3, as duas gravações ainda ficam na mesma transação do banco.

EXEMPLO 2 — O MAIS IMPORTANTE

# MoverEstagio: carregar, delegar, salvar

```
def executar(self, orcamento_id, novo_estagio):
    orcamento = obter_orcamento(self._orcamentos, orcamento_id)
    orcamento.mover_estagio(novo_estagio, hoje=self._relogio().date())
    self._orcamentos.salvar(orcamento)
    return orcamento
```

Só três linhas de trabalho. Se o id não existir, obter_orcamento lança OrcamentoNaoEncontrado (vira 404 na API).




**O teste de RNF04:**

```
with pytest.raises(TransicaoInvalida):
    use_case.executar(orcamento_id, E.FECHADO)          # pular etapa
assert orcamentos.salvamentos == salvamentos_antes      # nada foi gravado
```

# ConsultarOrcamentos: o painel Kanban




```
CardOrcamento(
  id
  cliente_id
  cliente_nome      "Maria Souza"
  estagio           NOVO_CONTATO
  valor             Decimal | None
  data_abertura     2026-09-15
  qtd_interacoes    2
)
```

Cada card é uma projeção enxuta: traz só a quantidade de interações, não as interações em si (RNF01).

As 5 colunas sempre aparecem, mesmo vazias (como PERDIDO acima). Dentro de cada coluna, quem espera há mais tempo vem primeiro (data de abertura crescente).

# Métricas do funil e histórico do cliente

## `ConsultarMetricas`

**taxa = FECHADO ÷ (FECHADO + PERDIDO)**

| | |
|---|---|
| 3 fechados, 1 perdido | **75,00%** |
| 1 fechado, 2 perdidos | **33,33%** |
| só orçamentos em aberto | **None → tela mostra "—"** |

calcular_metricas() é uma função pura: recebe uma lista de orçamentos e devolve números. Testada sem repositório nenhum.

## `ConsultarHistoricoCliente`

**09:00 · MENSAGEM** — Orçamento A
primeiro contato

**10:00 · REUNIÃO** — Orçamento A
visita ao cliente

**11:00 · LIGAÇÃO** — Orçamento B
novo pedido

Junta as interações de todos os orçamentos do cliente e ordena por data/hora (RN08). Cada item diz de qual orçamento veio, para a tela agrupar.

Por que a taxa não é zero quando nada foi concluído? Porque zero diria "você perdeu todos" — e ainda não houve nenhum resultado. Indefinido é a resposta honesta (RN07).

DUAS PEÇAS DE APOIO

# O relógio injetado e o “não encontrado”

## Relógio injetado

```
type Relogio = Callable[[], datetime]

# produção
lambda: datetime.now(UTC)

# testes: relógio parado
lambda: AGORA   # 15/09/2026 14:30
```

O caso de uso pergunta “que horas são?” ao relógio que recebeu. Nos testes o relógio está parado — por isso dá para afirmar a data exata de fechamento ou de uma interação.

## Não encontrado ≠ regra violada

**422** — `ErroDominio`

TransicaoInvalida, FormatoInvalido... O pedido faz sentido, mas a regra não deixa.

**404** — `RecursoNaoEncontrado`

ClienteNaoEncontrado, OrcamentoNaoEncontrado. O id pedido não existe.

Ficam em application/exceptions.py. Na Parte 4, a API converte cada uma no status HTTP certo.

RESULTADO

# Como a Parte 2 terminou

**160**

**testes passando**

em cerca de 0,3 segundo

**46**

**testes novos**

37 de casos de uso · 9 de contrato

**0**

**usos de banco nos testes**

tudo em memória (RNF06)

**8**

**casos de uso**

4 comandos · 4 consultas

Critério de pronto atingido: pytest verde, mypy --strict e ruff limpos, transição recusada não chama salvar() e o teste de arquitetura continua passando para application/.

MÃO NA MASSA

# Como explorar você mesmo

```
cd backend

# só os testes dos casos de uso, com o nome de cada um
.venv\Scripts\python.exe -m pytest tests/unit/application -v

# só os testes que têm "mover_estagio" no nome
.venv\Scripts\python.exe -m pytest -k mover_estagio -v
```

## Ordem sugerida de leitura

1. **`orcamento_repository.py`**
   a interface (domain/)
2. **`em_memoria.py`**
   o plugue em memória
3. **`mover_estagio.py`**
   o caso de uso central
4. **`test_mover_estagio.py`**
   os testes dele

## Exercício para fixar

1. Em src/application/use_cases/mover_estagio.py, troque a ordem: chame self._orcamentos.salvar(orcamento) ANTES de orcamento.mover_estagio(...).

2. Rode pytest -k mover_estagio -v. Os testes ficam vermelhos: o contador de salvamentos mudou mesmo com a transição recusada, e o estágio salvo é o antigo.

3. Desfaça a mudança (Ctrl+Z) e rode de novo: tudo verde. É assim que os testes protegem o RNF04.

PARA REVISAR

# Glossário rápido e próximo passo

| | |
|---|---|
| **Caso de uso** | Roteiro de uma ação do usuário; orquestra, não decide. |
| **Interface (porta)** | Contrato sem implementação: a “tomada”. Fica no domínio. |
| **Adapter** | Implementação concreta de uma porta: em memória, Postgres. |
| **Inversão de dep.** | O caso de uso depende da interface, não do banco. |
| **Repositório** | Guarda e busca agregados: salvar, buscar_por_id... |
| **Comando × consulta** | Comando muda dados e salva; consulta só lê. |

PRÓXIMO PASSO

## Parte 3 — Persistência

- Models SQLAlchemy e migration inicial com Alembic.
- Repositórios Postgres implementando as mesmas interfaces.
- Os testes de contrato rodando também contra o banco.
- Antes: Python 64 bits e Docker ou PostgreSQL instalados.
- A decidir: rota para abrir um 2º orçamento de um cliente (RF08).